

Berry TUI — Claude Code Workflow Visualizer

Product Specification v0.1

Vision

Berry is a terminal UI layer that wraps Claude Code’s agent execution and transforms it into a **living, watchable, beautiful experience**. Instead of a wall of log output, Berry renders your agent’s work as a real-time animated scene — with a waterfall of tool calls, a reactive character companion, and a fully customizable theme ecosystem.

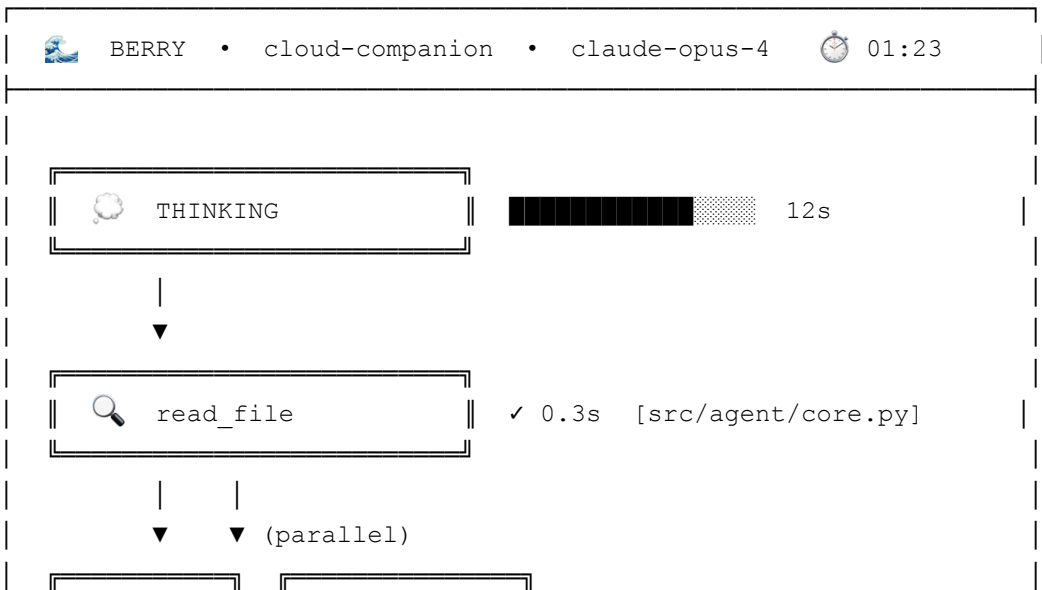
The core thesis: **you’re sitting there anyway**. Permission prompts, long inference runs, multi-file edits — these are moments of attention. Berry makes them worth watching.

Core Concepts

1. The Waterfall View

The primary visualization model — borrowed from LangFuse/AgentOps tracing but designed for a terminal and optimized for *aesthetics over exhaustiveness*.

Each agent action flows downward like a stream:





Branches visualize parallel tool calls. Depth represents nesting. Completed nodes show duration and status. Failed nodes render in theme's error color.

2. The Companion Character

A small animated character lives in the footer or sidebar. It **reacts emotionally** to what's happening:

Event	Companion Behavior
Thinking / inference	Pacing, tapping chin, "hmm..." bubble
Reading files	Holding tiny book, reading animation
Writing files	Typing at tiny desk
Permission request	👁️ Eyes wide, hands raised, waiting
Permission granted	Happy dance, sparkle burst
Permission denied	Slumps shoulders
Test passing	Jumping, confetti
Test failing	Face-palm, smoke effect
Long wait (>30s)	Falls asleep, zzz bubble
Completion	Bow, curtain drop
Error	Exploding then reforming

Characters are swappable per theme. Default character: **Berry** (a small friendly berry-colored sprite).

3. The Permission Theater

Permission prompts are currently the most awkward moment in Claude Code — they

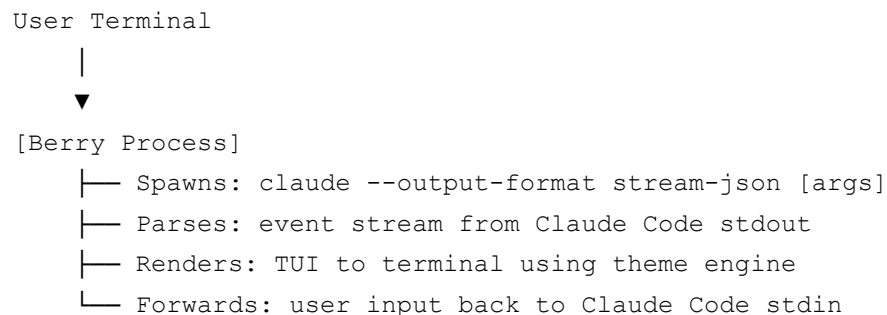
interrupt flow and feel like a system dialog. Berry transforms this:

- The UI dims and narrows focus to the permission panel
 - Companion character runs to center stage, looks nervous
 - The request is presented as a clear, readable card with context:
 - What is being requested
 - Which file/resource
 - Why (extracted from Claude's preceding reasoning if available)
 - Risk level indicator (read = blue, write = yellow, exec = orange, delete = red)
 - Animated countdown ring (configurable, default: none — user must actively decide)
 - Large, clear keybinding indicators
-

Architecture

How Berry Hooks Into Claude Code

Berry operates as a **wrapper process** around Claude Code, capturing and parsing its output stream in real-time:



Berry uses Claude Code's `--output-format stream-json` flag to receive structured events. It does not patch or modify Claude Code itself.

Primary event types consumed:

- `system` — session start, model info
- `assistant` — thinking blocks, text responses
- `tool_use` — tool invocations with parameters

- `tool_result` — tool results with status/duration
- `permission_request` — requires user interaction
- `result` — session end, cost summary

Data Model

```
interface AgentTrace {
  sessionId: string;
  model: string;
  startTime: number;
  nodes: TraceNode[];
  totalCost: number;
  totalTokens: TokenUsage;
}

interface TraceNode {
  id: string;
  parentId: string | null;
  type: 'thinking' | 'tool_use' | 'tool_result' | 'permission' | 'text';
  label: string;
  status: 'running' | 'complete' | 'failed' | 'pending';
  startTime: number;
  endTime?: number;
  metadata: Record<string, unknown>;
  children: TraceNode[];
}
```

Theme System

Theme Package Structure

```
my-theme/
├─ theme.json           # Required: colors, metadata, layout config
├─ character/
│   └─ frames/         # ASCII/Unicode animation frames
│       └─ idle.txt
│       └─ thinking.txt
│       └─ happy.txt
│       └─ ...
│   └─ character.json  # Frame timing, state machine definition
└─ waterfall/
```

```

|   ├── particles.json # Particle characters, colors, physics
|   └── nodes.json     # Node box styles per event type
├── sounds/            # Optional: terminal bell sequences or OSC codes
|   └── events.json
└── README.md

```

theme.json Schema

```

{
  "name": "Cyberpunk Neon",
  "version": "1.0.0",
  "author": "you",
  "description": "Neon-soaked aesthetic. Pink and cyan on black.",

  "colors": {
    "background": "#0a0a0a",
    "primary": "#ff2d78",
    "secondary": "#00f5ff",
    "accent": "#f5a623",
    "success": "#39ff14",
    "warning": "#ff9500",
    "error": "#ff3b30",
    "muted": "#333333",
    "text": "#e0e0e0",
    "text_dim": "#666666"
  },

  "typography": {
    "node_border": "double", // single | double | rounded | heavy | ascii
    "progress_fill": "█",
    "progress_empty": "░",
    "branch_vertical": "|",
    "branch_horizontal": "-",
    "branch_fork": "└",
    "branch_corner": "└",
    "flow_arrow": "▼"
  },

  "layout": {
    "waterfall_position": "center", // center | left | full
    "companion_position": "bottom-right", // bottom-right | bottom-left | sidebar
    "header": true,
    "footer": true,
    "show_cost": true,
    "show_tokens": true
  },
}

```

```

"waterfall": {
  "particles": true,
  "particle_chars": [".", "•", "o", "◦"],
  "particle_color": "secondary",
  "flow_speed": "medium",          // slow | medium | fast
  "node_glow": true
},

"animations": {
  "fps": 12,
  "node_enter": "slide-down",      // slide-down | fade | pop | none
  "node_complete": "flash",        // flash | checkmark | none
  "node_error": "shake"            // shake | flash | none
}
}

```

Character State Machine (character.json)

```

{
  "states": {
    "idle": {
      "frames": ["idle_0.txt", "idle_1.txt", "idle_2.txt"],
      "fps": 2,
      "loop": true,
      "transitions": {
        "on_thinking": "thinking",
        "on_tool_start": "working"
      }
    },
    "thinking": {
      "frames": ["think_0.txt", "think_1.txt", "think_2.txt", "think_3.txt"],
      "fps": 4,
      "loop": true,
      "speech_bubble": ["hmm...", "🤔", "aha!", "wait..."],
      "bubble_interval_ms": 3000
    },
    "permission_wait": {
      "frames": ["perm_0.txt", "perm_1.txt"],
      "fps": 2,
      "loop": true,
      "speech_bubble": ["please?", "just say yes 🙏"]
    },
    "celebrating": {
      "frames": ["celebrate_0.txt", "celebrate_1.txt", "celebrate_2.txt"],
      "fps": 8,

```

```
        "loop": false,
        "on_complete": "idle"
    }
}
}
```

Built-in Themes (v1.0)

Theme	Character	Vibe
<code>berry</code> (default)	Berry the sprite	Friendly, colorful, warm
<code>waterfall</code>	Koi fish	Calm, zen, flowing water animations
<code>cyberpunk</code>	Glitch ghost	Neon, edgy, matrix rain particles
<code>forest</code>	Forest sprite	Earthy greens, nature sounds
<code>space</code>	Tiny astronaut	Stars, cosmic, quiet wonder
<code>minimal</code>	None	Pure information, no character
<code>retro</code>	ASCII robot	CGA colors, pixel aesthetic
<code>catppuccin</code>	Cat	Pastel, soft, community favorite

Information Architecture

What Berry Displays

Header Bar:

- Session name / project name
- Model name + version
- Elapsed time (HH:MM:SS)
- Connection status indicator

Main Waterfall Panel:

- Scrollable trace of all nodes

- Each node shows: type icon, label, status, duration
- Expandable nodes (press Enter) show full metadata
- Color-coded by status and type
- Parallel branches rendered side-by-side

Node Detail View (on selection):

- Full tool parameters (formatted)
- Full tool result or error
- Token cost for this node
- Timestamp

Footer Bar:

- Companion character
- Token counter (input / output / cache)
- Estimated cost (running total)
- Keyboard shortcut hints

Permission Panel (overlay):

- Full-screen dim of background
- Permission card: type, target, context, risk level
- Companion moves to center
- Large Y/N indicators

Keyboard Shortcuts

Key	Action
↑/↓ or j/k	Navigate nodes
Enter	Expand/collapse node detail
t	Cycle themes
c	Toggle companion character

s	Toggle sounds
f	Toggle fullscreen waterfall
e	Export trace (JSON / Markdown)
/	Search trace
?	Help overlay
q	Quit (with confirmation)

Plugin / Extension API

Beyond themes, Berry supports **plugins** — code that hooks into the rendering pipeline.

```
// Plugin interface
interface BerryPlugin {
    name: string;
    version: string;

    // Lifecycle hooks
    onSessionStart?(session: AgentTrace): void;
    onNodeAdded?(node: TraceNode): void;
    onNodeComplete?(node: TraceNode): void;
    onPermissionRequest?(request: PermissionRequest): void;
    onSessionEnd?(session: AgentTrace): void;

    // Render hooks (return null to use default rendering)
    renderNode?(node: TraceNode, theme: Theme): string | null;
    renderHeader?(session: AgentTrace, theme: Theme): string | null;
    renderFooter?(session: AgentTrace, theme: Theme): string | null;
}
```

Example plugin ideas:

- `berry-plugin-sound` : Play terminal sounds on events (using OSC sequences or terminal bell patterns)
- `berry-plugin-slack` : Post a Slack message when Claude Code completes a session
- `berry-plugin-stats` : Track personal analytics across sessions (time saved, cost per task, etc.)

- `berry-plugin-ai-commentary` : Use a second, tiny model to provide sardonic real-time commentary on what Claude is doing
 - `berry-plugin-speedrun` : Show a timer and compare to your personal best for similar tasks
-

Configuration

`.berryrc` (project or global)

```
# ~/.berryrc or <project>/.berryrc

theme: berry                # Theme name or path to theme dir
character: true             # Show companion character
sounds: false              # Audio feedback
fps: 12                    # Animation frame rate
cost_display: running_total # none | running_total | per_node | both
token_display: true
export_on_complete: false  # Auto-save trace on session end
export_format: json        # json | markdown
export_path: ~/.berry/traces/

plugins:
- berry-plugin-stats
- ~/my-custom-plugin

# Override specific theme values
theme_overrides:
  colors:
    primary: "#your-brand-color"
  companion_position: bottom-left
```

Technical Stack

Recommended Implementation

Language: TypeScript / Node.js

Rationale: Claude Code is already Node, easy to distribute via npm, large ecosystem of TUI libs

TUI Framework: [ink](#) (React for CLIs) or [blessed](#)

ink preferred for its React model and composability with themes

Animation: Custom frame loop using `setInterval` + ANSI escape sequences

Unicode/ANSI: Full support required. Degrade gracefully for non-Unicode terminals.

Distribution:

```
npm install -g @berry-tui/core      # Core
npm install -g @berry-tui/theme-xxx # Theme packs
```

Usage:

```
berry "implement the search feature in src/search.py"
berry --theme cyberpunk --no-character "fix the bug"
berry run claude [all normal claude flags]      # Transparent passthrough
```

MVP Scope (v0.1)

For an initial implementation to prove the concept:

1.  Basic waterfall rendering of tool calls in real-time
2.  Header with model + elapsed time + token counter
3.  Node status indicators (running / complete / failed)
4.  Single default theme (`berry`)
5.  Basic companion character (3–4 states: idle, thinking, working, done)
6.  Permission request overlay with clear context
7.  Keyboard navigation of trace nodes
8.  `theme.json` loading from disk (no plugins yet)

Not in MVP:

- Plugin API
- Sounds
- Multiple built-in themes
- Export

- Search
-

Inspiration & References

- **LangFuse** — the waterfall mental model
 - **htop** — information density done right in a terminal
 - **lazygit** — proof that TUIs can be genuinely *delightful* developer tools
 - **Tamagotchi** — reactive characters that respond to your actions
 - **Pokémon battle UI** — clear status, big actionable moments
 - **Studio Ghibli** — proof that watching something work can itself be beautiful
-

Open Questions for Implementation

1. How to handle very wide parallel branches on narrow terminals?
 2. Should Berry support mouse interaction, or keyboard-only?
 3. What's the right granularity for companion state — per tool call or per logical step?
 4. Should theme distribution go through npm, or a Berry-specific registry?
 5. How to handle streaming text output from Claude (long text responses mid-task)?
 6. Is there an opportunity to connect Berry to AgentOps-Core telemetry directly?
-

"The best developer tools don't just get out of the way — they make the work feel alive."